

DRL Portfolio

Un agent de trading par apprentissage par renforcement profond :
mon cheminement, mes erreurs, mes corrections, mes résultats

Rapport pédagogique — version du 3 juillet 2026

Martin Chassaing
IMT Atlantique & Université Paris Dauphine

Résumé

Ce rapport retrace la construction d'un agent de trading par *Deep Reinforcement Learning* (PPO) sur données journalières réelles, en suivant le cheminement réel du projet : ce que j'ai essayé, ce qui n'a pas fonctionné, pourquoi, et ce que chaque constat m'a poussé à faire ensuite. Sur un test set jamais vu à l'entraînement (février 2021 → décembre 2022), l'agent multi-actifs réalise +23,8 % contre -3,7 % pour le Buy & Hold (alpha +27,5 %), avec un alpha positif sur les cinq actifs testés. Mais la validation *walk-forward* (25 cellules année×actif out-of-sample) que j'ai menée ensuite relativise ce chiffre : l'agent a un profil *défensif régime-dépendant* — rendement absolu positif 4 années sur 5 et alpha fortement positif dans les marchés baissiers, mais retard net sur le B&H dans les fortes hausses. Chaque étape est reliée aux notions d'économétrie (stationnarité, GARCH, biais de look-ahead, validation out-of-sample) et de gestion de portefeuille (alpha de Jensen, Sharpe, Sortino, CVaR, drawdown). Enfin, deux stress-tests que je me suis imposés — sensibilité aux frais et ablation du kill-switch — montrent que l'essentiel de l'alpha 2022 provient de la règle de stop : un alpha de *gestion du risque* plus que de signal, distinction que j'assume et documente. Projet strictement éducatif.

Table des matières

1	Objectif et démarche	2
2	Architecture du projet	3
3	Base théorique : du RL au trading	3
3.1	Le trading comme processus de décision markovien	3
3.2	PPO en deux idées	4
3.3	Single-agent, multi-actifs, multi-agents : ne pas confondre	4
4	Features : le pont avec l'économétrie	5
4.1	Pourquoi ma récompense est un alpha de Jensen par pas de temps	6
4.2	Métriques d'évaluation	7
5	Chronologie : ce qui ne marchait pas, ce que j'ai changé	7
5.1	Point de départ	7
5.2	Itération 1 — des tests, parce qu'un simulateur faux rend tout faux	7
5.3	Itération 2 — mon protocole d'évaluation mentait	8
5.4	Itération 3 — plus de cycles de marché	8
5.5	Itération 4 — le bug décisif : ma validation était corrompue	8
5.6	Récapitulatif chiffré	9

6 Résultats sur le split de référence	10
6.1 Test AAPL, février 2021 → décembre 2022	10
6.2 Généralisation cross-actifs	12
7 Du backtest unique au walk-forward	12
7.1 Pourquoi c'était la priorité (et pas la diffusion ou le MARL)	12
7.2 Protocole	13
7.3 Ce que j'avais prédit — et ce qui s'est passé	13
7.4 Interprétation financière : un profil, pas un alpha universel	14
7.5 Ce que ça ouvre	14
8 Stress-tests : les questions qu'un desk poserait	14
8.1 « Ton alpha survit-il à des frais réalistes ? »	14
8.2 « Que reste-t-il si j'enlève ton kill-switch ? »	15
8.3 Les questions auxquelles je n'ai pas encore de mesure	15
9 Overfitting, underfitting, régimes : le diagnostic complet	15
10 Limites honnêtes et travaux futurs	16
11 Glossaire minute	16
12 Reproduire les résultats	18
A Annexe mathématique	19
A.1 Du policy gradient à l'avantage	19
A.2 Estimation de l'avantage : GAE	19
A.3 La perte PPO complète	20
A.4 Le processus d'Ornstein-Uhlenbeck en détail	20

1 Objectif et démarche

Je suis parti d'une question simple : *un agent d'apprentissage par renforcement peut-il apprendre à battre une stratégie passive (Buy & Hold) sur données réelles, une fois les frais de transaction pris en compte ?*

À la fin du parcours, ma réponse tient en trois temps, et c'est ce cheminement que ce rapport documente :

1. mes premiers résultats semblaient bons mais étaient **invalides** — pas parce que le modèle était mauvais, mais parce que mon protocole d'évaluation était faux (observations non normalisées, frais incohérents, validation corrompte) ;
2. une fois le protocole corrigé et le modèle réentraîné, l'agent **bat nettement le B&H** sur mon split de test, et généralise sur cinq actifs ;
3. la validation walk-forward m'a ensuite appris que ce résultat était **partiellement un artefact de la période de test** : mon agent est défensif, brillant en marché baissier, à la traîne dans les fortes hausses.

La leçon transversale : en finance quantitative, *la qualité d'une conclusion vaut celle du protocole qui la produit*. J'ai passé plus de temps à fiabiliser l'évaluation qu'à optimiser le modèle — et c'est l'évaluation qui a tout changé.

2 Architecture du projet

Fichier	Rôle	Contenu
<code>data_loader.py</code>	Données	Téléchargement yfinance, feature engineering, normalisation <i>fittée sur le train uniquement</i> , split temporel 70/15/15, pipeline multi-actifs avec segments.
<code>environment.py</code>	Simulation	Environnement Gymnasium : 4 actions (Hold/Long/Flat/Short), comptabilité exacte des frais, kill-switch drawdown, épisodes confinés par actif.
<code>train.py</code>	Entraînement	PPO (Stable-Baselines3), 4 environnements parallèles, normalisation des observations (VecNormalize), sélection du meilleur modèle sur la validation.
<code>evaluate.py</code>	Évaluation	Protocole déterministe sur split complet, robustesse sur sous-fenêtres, Sharpe/Sortino/Calmar/CVaR, diagnostic d'overfitting, généralisation cross-actifs.
<code>walk_forward.py</code>	Validation	Réentraînements glissants (fenêtre croissante ancrée), une année de test 100 % out-of-sample par fold, grille année×actif.
<code>dashboard.py</code>	Visualisation	Application Streamlit interactive.
<code>tests/</code>	Qualité	67 tests pytest : pipeline de données, mécanique de trading, conformité Gymnasium, entraînements PPO courts réels, évaluation, walk-forward, dashboard.
<code>reports/</code>	Présentation	Ce rapport, les figures, <code>build_assets.py</code> (figures + dashboard statique <code>docs/index.html</code>).

Table 1 – Organisation du dépôt.

3 Base théorique : du RL au trading

3.1 Le trading comme processus de décision markovien

L'apprentissage par renforcement modélise un *processus de décision markovien* (MDP) : à chaque jour de bourse t , l'agent observe un état s_t , choisit une action a_t selon sa politique $\pi(a_t | s_t)$, reçoit une récompense r_t , et l'environnement transitionne vers s_{t+1} . L'objectif est de maximiser l'espérance de la somme actualisée des récompenses $\mathbb{E}[\sum_t \gamma^t r_t]$, avec $\gamma = 0,99$: l'agent « voit » environ $1/(1 - \gamma) = 100$ jours devant lui, un horizon cohérent avec du trading journalier.

Deux termes que j'emploierai partout : un **épisode** est une traversée de la simulation, du point de départ jusqu'à la fin des données ou au déclenchement du stop ; une **seed** est la graine du générateur pseudo-aléatoire — la fixer rend chaque expérience exactement reproductible (indispensable pour comparer deux versions du code à conditions identiques).

Observation. Une fenêtre glissante de $10 \text{ jours} \times 5 \text{ features de marché}$ (section 4), plus deux variables de portefeuille : la position courante (-1 short, 0 cash, $+1$ long) et le PnL latent. Soit $10 \times 5 + 2 = 52$ entrées. Remarque théorique honnête : les prix de marché ne sont évidemment pas markoviens dans une fenêtre de 10 jours — l'hypothèse MDP est une approximation, partiellement compensée par des features qui résument le passé (volatilité, momentum).

Actions. {Hold, Long, Flat, Short}. Le short est essentiel : sans lui, le mieux qu'un agent puisse faire dans un marché baissier est de *ne pas perdre* (rester cash) ; avec lui, il peut *gagner* pendant les baisses — c'est exactement ce qui se produit en 2022 (figure 3).

3.2 PPO en deux idées

PPO (*Proximal Policy Optimization*) est un algorithme de *policy gradient* : un réseau de neurones (ici volontairement petit, 2×64) produit directement la distribution sur les actions, améliorée par montée de gradient sur l'avantage estimé A_t (« cette action a-t-elle rapporté plus que ce que la valeur de l'état laissait espérer ? »). Sa spécificité est le *clipping* :

$$L(\theta) = \mathbb{E}_t [\min(\rho_t(\theta) A_t, \text{clip}(\rho_t(\theta), 1 - \varepsilon, 1 + \varepsilon) A_t)], \quad \rho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)},$$

qui borne l'ampleur de chaque mise à jour de politique (ici $\varepsilon = 0,15$). Sur des rendements financiers — un signal au ratio bruit/information écrasant — cette prudence n'est pas un détail d'implémentation : sans elle, une séquence chanceuse suffirait à faire basculer toute la politique.

Trois rouages qu'il faut savoir expliquer :

- **fonction de valeur** : un second réseau estime $V(s_t)$, « ce que cet état devrait rapporter en moyenne » ; l'avantage A_t mesure ce que l'action a rapporté *au-delà* de cette attente — c'est lui qui dit au gradient quoi renforcer ;
- **bonus d'entropie** (coefficient 0,05) : un petit terme qui récompense le fait de garder des probabilités d'action diversifiées ; sans lui, la politique se fige prématurément sur une action (typiquement « toujours Hold » ou « toujours Long ») avant d'avoir exploré les alternatives ;
- **normalisation des observations** (VecNormalize) : pendant l'entraînement, moyenne et variance *glissantes* de chaque composante de l'observation sont maintenues, et les observations sont standardisées à la volée. Distinct du scaler des features (section 4) : il couvre aussi position et PnL, et ses statistiques doivent être *sauvegardées avec le modèle* — un modèle nourri d'observations brutes à l'évaluation produit du non-sens, comme je l'ai appris à mes dépens (section 5.3).

3.3 Single-agent, multi-actifs, multi-agents : ne pas confondre

Trois notions que je tiens à distinguer précisément, parce que la confusion est fréquente (et que mon propre vocabulaire de projet peut la créer) :

- **Single-agent, mono-actif** : un décideur, un actif (mon modèle « Single », entraîné sur AAPL seul). L'environnement est stationnaire du point de vue de l'agent — au sens où ses propres actions n'influencent pas les prix.
- **Single-agent, multi-actifs** : c'est mon modèle « Multi (5 tickers) » : *un seul* agent, entraîné sur les données de cinq actifs (chaque épisode se déroule sur un actif, tiré au hasard). Ce n'est PAS du multi-agents : c'est de la *régularisation par diversité des données* — pour être bon sur AAPL, MSFT, GOOGL, SPY et TSLA à la fois, il faut des règles générales, pas une trajectoire apprise par cœur.
- **Multi-agents (MARL)** : plusieurs politiques *apprennent simultanément* et interagissent. La difficulté fondamentale : du point de vue de chaque agent, l'environnement devient *non stationnaire*, puisque les autres agents changent en apprenant — les garanties de convergence du RL classique sautent. La réponse standard est le CTDE (*centralized training, decentralized execution*) : un critique centralisé qui voit tout pendant l'entraînement, des politiques décentralisées à l'exécution (MAPPO, MADDPG). C'est la phase suivante de ma roadmap : ajouter un *adversaire de marché* qui apprend à créer les scénarios les plus défavorables (chocs de liquidité, pics de volatilité) — de l'*adversarial training* pour forcer la robustesse, précisément la faiblesse que le walk-forward va révéler (section 7).

4 Features : le pont avec l'économétrie

Chaque feature que je donne à l'agent répond à un problème identifié en économétrie des séries temporelles.

Feature	Définition	Notion d'économétrie associée
log_returns	$r_t = \ln(P_t/P_{t-1})$	Les prix sont $I(1)$ (non stationnaires, cf. test ADF) : je travaille sur la série différenciée, stationnaire en moyenne, comme pour toute modélisation ARMA.
volatility	écart-type glissant de r_t sur 20 jours	Estimateur naïf de σ_t : proxy du <i>volatility clustering</i> modélisé par GARCH(1,1), $\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2$ (Engle, Bollerslev). L'agent observe le régime de volatilité sans estimer le modèle.
rsi	oscillateur borné $[0, 1]$ sur 14 jours	Signal de sur-achat/sur-vente : pari de <i>retour à la moyenne</i> local — l'esprit d'un processus d'Ornstein-Uhlenbeck, $dX_t = \kappa(\mu - X_t) dt + \sigma dW_t$, qui rappelle la variable vers sa moyenne μ à vitesse κ , sans son estimation paramétrique.
macd_norm	$(\text{EMA}_{12} - \text{EMA}_{26})/\text{EMA}_{26}$	Détection de tendance par différence de moyennes mobiles exponentielles — un filtre linéaire de la série des prix.
momentum_5	rendement sur 5 jours	Anomalie <i>momentum</i> (Jegadeesh & Titman, 1993) : autocorrélation positive des rendements à moyen terme, une des violations les plus documentées de l'efficience faible.

Table 2 — Correspondance features $\leftrightarrow$ économétrie.

Le biais de look-ahead, version machine learning. En économétrie, on n'évalue jamais un modèle sur l'échantillon qui a servi à l'estimer. Ici, l'équivalent se niche dans un détail : la *normalisation*. Le **RobustScaler** (centrage médiane, réduction par écart interquartile — robuste aux queues épaisses des rendements) est **fitté sur le train uniquement**, puis appliqué tel quel à la validation et au test. Le fitter sur tout l'échantillon reviendrait à injecter dans les observations de 2015 des statistiques calculées avec les données de 2022 : de l'information future. Un test unitaire verrouille ce point en comparant les paramètres du scaler à ceux d'un scaler fitté manuellement sur le train seul.

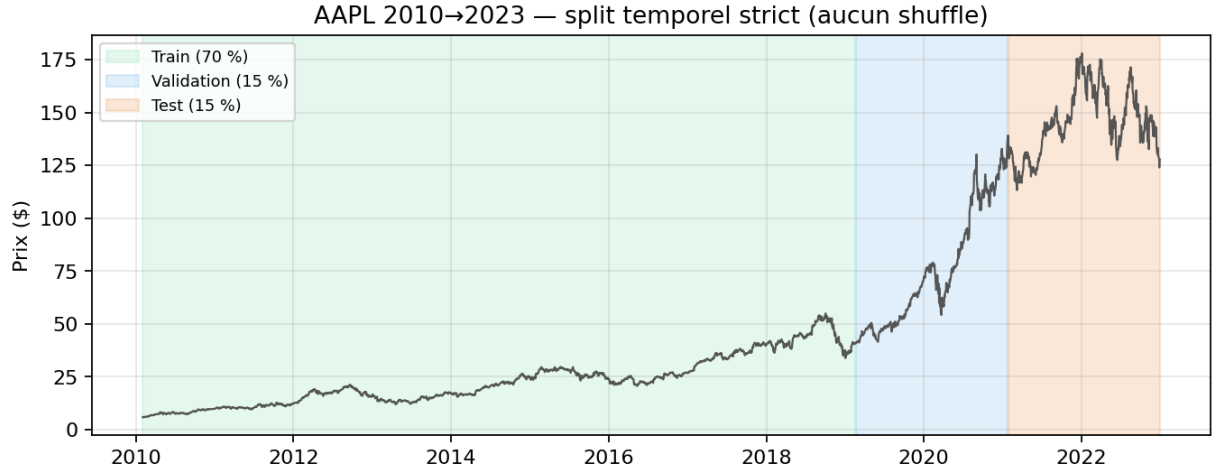


Figure 1 — Split temporel strict 70/15/15 sur AAPL 2010→2023. Le train couvre plusieurs cycles (2011, 2015, 2018), la validation contient le krach COVID et sa reprise, le test enchaîne bull 2021 et bear 2022. C’est l’équivalent de l’évaluation out-of-sample d’un modèle économétrique.

Zoom : le retour à la moyenne et le processus d’Ornstein-Uhlenbeck. C’est le modèle canonique du retour à la moyenne en temps continu :

$$dX_t = \kappa (\mu - X_t) dt + \sigma dW_t,$$

où μ est le niveau de long terme, $\kappa > 0$ la *vitesse de rappel* (plus κ est grand, plus vite tout écart se résorbe) et σdW_t le choc brownien. Deux quantités à savoir manier : la **demi-vie** d’un écart, $t_{1/2} = \ln 2 / \kappa$ (« en combien de jours la moitié de l’écart à μ est-elle effacée ? »), et le lien direct avec l’économétrie : *discretisé, un OU est exactement un AR(1)*, $X_{t+1} = \mu(1 - \varphi) + \varphi X_t + \varepsilon_t$ avec $\varphi = e^{-\kappa \Delta t}$ — on estime donc κ par une simple régression MCO de X_{t+1} sur X_t (dérivation complète en annexe A.4). Dans ce projet, je n’estime pas d’OU : le RSI en est le cousin non paramétrique (un oscillateur borné qui signale les écarts extrêmes susceptibles de se résorber). Mais le cadre OU est celui que j’utiliserais pour trader un *spread* de paires cointégrées — le projet suivant sur ma liste — et c’est le vocabulaire attendu en entretien dès qu’on dit « mean reversion ».

4.1 Pourquoi ma récompense est un alpha de Jensen par pas de temps

Mon premier réflexe avait été une récompense de type Sharpe par pas. Symptôme observé : l’agent devenait « toujours long ». Diagnostic : comme le marché monte en moyenne sur la période, *suivre le marché* suffit à collecter du rendement — la récompense ne distinguait pas « profiter du marché » de « battre le marché ». J’ai donc changé la cible : à chaque pas,

$$r_t = \underbrace{\left(\frac{V_t - V_{t-1}}{V_{t-1}} - \frac{P_t - P_{t-1}}{P_{t-1}} \right)}_{\text{alpha du pas } t} \times 100 - \text{pénalité de frais} - 0,05 \times \text{drawdown},$$

où V_t est la valeur du portefeuille et P_t le prix. Le premier terme est un **alpha de Jensen** dans le cas particulier $\beta = 1$, $R_f = 0$:

$$\alpha_J = R_p - [R_f + \beta (R_m - R_f)] \xrightarrow{\beta=1, R_f=0} R_p - R_m.$$

Être long quand ça monte ne rapporte plus rien (alpha ≈ 0) ; être cash ou short quand ça baisse rapporte ; rater une hausse coûte. L’agent est payé pour *timer*, pas pour *suivre*. Le facteur $\times 100$ est une mise à l’échelle : PPO apprend bien avec des récompenses d’ordre de grandeur ~ 1 , alors qu’un return journalier brut ($\sim 0,01$) donnerait un signal de gradient trop faible.

Les deux pénalités sont des outils de gestion classiques : les **frais** (0,1 % par transaction, débités du portefeuille *et* pénalisés dans la récompense) découragent l'overtrading ; la pénalité de **drawdown** introduit une aversion au risque asymétrique, complétée par un *kill-switch* : tout épisode dont le drawdown dépasse 25 % s'arrête — l'équivalent d'un stop-loss de mandat.

4.2 Métriques d'évaluation

Toutes viennent du cours de gestion de portefeuille :

- **Sharpe** = $\frac{\bar{r}}{\sigma} \sqrt{252}$: rendement par unité de risque total (critère moyenne-variance de Markowitz) ;
- **Sortino** : idem avec l'écart-type des seuls rendements négatifs (semi-variance) — on ne pénalise pas la volatilité haussière ;
- **Max drawdown** : perte maximale depuis un pic, le risque « vécu » ;
- **Calmar** = rendement annualisé / max drawdown ;
- **CVaR₉₅** (Expected Shortfall) = $\mathbb{E}[r \mid r \leq \text{VaR}_{95}]$, où la VaR_{95} est simplement le quantile à 5 % des rendements journaliers (« le seuil de perte dépassé un jour sur vingt »). La CVaR répond à la question suivante : *quand* ce seuil est franchi, combien perd-on en moyenne ? Contrairement à la VaR, c'est une mesure cohérente (sous-additive), celle de la réglementation bancaire récente.

5 Chronologie : ce qui ne marchait pas, ce que j'ai changé

C'est le cœur du rapport. Chaque itération suit le même schéma : *symptôme observé* → *diagnostic* → *pourquoi c'est grave financièrement* → *décision* → *résultat mesuré* → *ce que ça ouvre*.

5.1 Point de départ

Deux modèles PPO entraînés sur 2018–2023 (un mono-actif AAPL, un multi-actifs), récompense alpha, observations normalisées par **VecNormalize**. Mon évaluation d'alors donnait : Single battant le B&H sur 3 tirages sur 5 (alpha moyen +8,5 % ± 10,2), Multi négatif (−4,7 %, 0/5).

Trois symptômes me gênaient : (i) une variance énorme entre tirages (±10 à 15 points — comment conclure quoi que ce soit ?), (ii) un alpha d'entraînement de plusieurs milliers de pourcents (personne ne croit ça : mémorisation), (iii) un Multi « toujours long », alors que sa raison d'être était justement de généraliser. *Avant de toucher au modèle, j'ai décidé de fiabiliser tout ce qui produit les chiffres.*

5.2 Itération 1 — des tests, parce qu'un simulateur faux rend tout faux

En finance simulée, un test ne vérifie pas que « le code tourne » : il vérifie que *l'économie du simulateur est juste*. J'ai écrit une suite de tests qui fixe des vérités comptables : un aller-retour long à prix constant doit coûter exactement $(1 - c)^2$ du capital ; un short doit gagner quand le prix baisse ; le kill-switch doit se déclencher au bon seuil ; la normalisation ne doit rien voir du futur.

Résultat : les tests ont immédiatement trouvé une asymétrie — **les frais d'ouverture d'un short n'étaient pas débités du portefeuille** (ils réduisaient seulement la taille de position), contrairement au long. Un simulateur qui sous-facture le short gonfle mécaniquement toute stratégie qui shorte : exactement le genre de biais qui fabrique de faux alphas. Corrigé, verrouillé par un test de symétrie.

Ce que ça ouvre : avec un simulateur fiable, je peux maintenant regarder si mes chiffres d'évaluation, eux, sont crédibles.

5.3 Itération 2 — mon protocole d'évaluation mentait

Trois défauts faussaient les *conclusions*, indépendamment de la qualité des modèles :

1. **Observations non normalisées à l'évaluation.** Le modèle est entraîné sur des observations standardisées ; deux scripts prédisaient sur observations *brutes*. C'est comme estimer une régression sur variables centrées-réduites puis prédire sur variables en niveau : les sorties n'ont aucun sens.
2. **Frais d'évaluation $\neq$ frais d'entraînement** (0,2 % vs 0,1 %) : j'évaluais l'agent dans un monde deux fois plus coûteux que celui où il avait appris à trader.
3. **Évaluation sur fenêtres aléatoires uniquement**, d'où la variance de ± 15 points. Pire : un épisode arrêté par le kill-switch se termine *par construction* à son point bas, et était comparé à un B&H calculé sur une fenêtre différente — des alphas incomparables entre modèles.

Décision : un protocole de référence **déterministe sur le split complet** (départ fixe, politique déterministe, reproductible au centime) ; si le kill-switch se déclenche, la stratégie est réputée liquidée et **reste en cash jusqu'à la fin de la fenêtre** — tous les modèles et le B&H sont ainsi comparés sur le même horizon. Les fenêtres aléatoires deviennent une mesure *complémentaire* de robustesse au point d'entrée.

Ce que ça ouvre : des chiffres enfin stables — et donc la possibilité de voir qu'il restait un vrai problème de données.

5.4 Itération 3 — plus de cycles de marché

Symptôme : mon test d'origine (avril-décembre 2022) était un bear pur. Interprétation financière : sur une telle période, un agent simplement *peureux* paraît génial — biais de période classique du backtesting. Décision : réentraîner sur 2010→2023 ($2,6\times$ plus de données, plusieurs corrections dans le train, validation contenant le COVID) avec un test 2021–2022 qui enchaîne hausse et baisse.

Ce que ça ouvre : un cadre assez propre pour que le dernier symptôme — le Multi toujours décevant — ne puisse plus s'expliquer que par une vraie cause.

5.5 Itération 4 — le bug décisif : ma validation était corrompue

Symptôme : après réentraînement, le Multi restait mauvais (alpha test négatif, biais long), et les logs d'entraînement montraient des drawdowns de validation de 45 à 50 % — physiquement impossibles avec un kill-switch à 25 %.

Diagnostic : le pipeline multi-actifs concatène les données des cinq tickers. Un identifiant de segment garantissait que les épisodes d'*entraînement* ne franchissent pas la frontière entre deux actifs... mais pas ceux de la *validation*. Un épisode de validation pouvait donc passer du dernier jour de GOOGL ($\sim 1\,000$ \$) au premier jour de SPY (~ 300 \$) : un krach fictif de -70% en un jour.

Pourquoi c'est grave : c'est précisément sur cette validation que l'EvalCallback choisit le *best model* conservé à la fin. **Je sélectionnais mon meilleur modèle sur du bruit** — l'équivalent économétrique d'un critère de choix de modèle calculé sur un échantillon pollué.

Décision et résultat : une ligne de correction (segments sur les trois splits), un réentraînement — le Multi passe à **+27,5 % d'alpha** sur le test, avec un alpha positif sur **5/5 actifs** (figure 5) et une répartition d'actions enfin équilibrée. La leçon que je retiens : *la sélection de modèle vaut ce que vaut le signal de validation.*

La preuve propre : une ablation contrôlée. Le tableau 4 a une faiblesse méthodologique que je préfère lever moi-même : entre « départ » et « final », j'ai changé plusieurs choses (dates, protocole, frais du short, bug de validation) — c'est un *confondage*, on ne sait pas quelle cause

a produit quel effet. J’ai donc mené l’expérience contrôlée : réentraîner le Multi **en ne réintroduisant que le bug** (validation sans segments), tout le reste strictement identique (données 2010–2023, seed, 800 000 pas, protocole d’évaluation final).

Multi, toutes choses égales par ailleurs	Validation corrompue	Validation propre
Alpha test AAPL	−2,4 %	+27,5 %
Actifs à alpha positif	3/5	5/5
Alpha cross-actifs moyen	+10,4 %	+17,2 %

Table 3 – Ablation contrôlée du bug de validation : effet causal isolé $\approx +30$ points d’alpha sur AAPL. Détail du modèle corrompu : TSLA +37 %, GOOGL +20,5 %, MSFT +5,9 %, AAPL −2,4 %, SPY −8,8 %.

Le détail est aussi instructif que la moyenne : le modèle sélectionné sur une validation bruitée n’est pas *uniformément* mauvais — il est *erratique* (excellent sur TSLA, négatif sur AAPL et SPY). C’est exactement ce que prédit la théorie : sélectionner sur du bruit revient à tirer un checkpoint *au hasard* parmi les versions du réseau — une loterie, dont on gagne certains tickets. La validation propre ne rend pas le modèle plus intelligent ; elle rend sa sélection *systématique* au lieu d’aléatoire.

Ce que ça ouvre : des résultats crédibles sur UN split. Reste la question qu’un professionnel poserait immédiatement : et sur les autres périodes ? (section 7).

5.6 Récapitulatif chiffré

Indicateur (test set)	Départ (2018-23, protocole v0)	Après itérations 1–4 (2010-23, protocole final)	
Alpha Single	+8,5 % $\pm$ 10,2	+19,2 %	↑
Alpha Multi	−4,7 % $\pm$ 2,8	+27,5 %	↑
Fenêtres où Single > B&H	3/5	5/5	↑
Fenêtres où Multi > B&H	0/5	5/5	↑
Actifs à alpha positif (Multi)	1/5	5/5	↑
Répartition des actions (Multi)	biais long	équilibrée (short 23 %)	↑

Table 4 – Avant/après. Les protocoles diffèrent entre colonnes *parce que corriger le protocole fait partie des améliorations* ; chaque chiffre est mesuré avec le protocole en vigueur à son étape (section 5.3). L’effet du bug de validation *seul*, toutes choses égales par ailleurs, est isolé par l’ablation contrôlée du tableau 3.

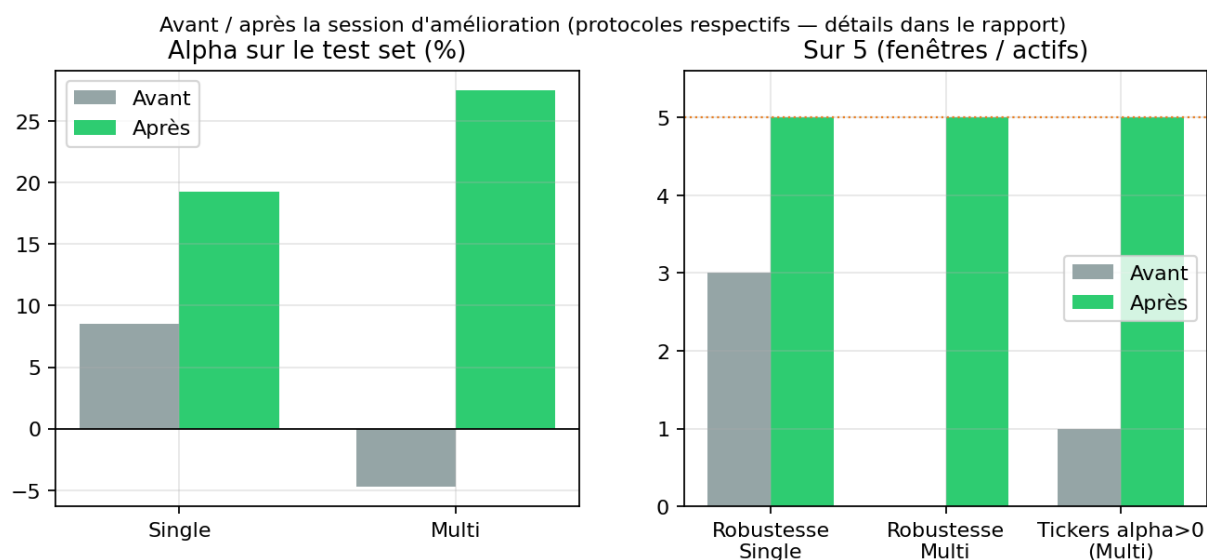


Figure 2 – Synthèse graphique du tableau 4.

6 Résultats sur le split de référence

6.1 Test AAPL, février 2021 → décembre 2022

Modèle	Return	Alpha	MaxDD	Sharpe	Sortino	Calmar	CVaR ₉₅
Single (AAPL)	+15,5 %	+19,2 %	26,2 %	0,44	0,57	0,30	-3,4 %
Multi (5 actifs)	+ 23,8 %	+ 27,5 %	25,2 %	0,57	0,77	0,47	-3,7 %
Buy & Hold (réf.)	-3,7 %	—	30,3 %	—	—	—	—

Table 5 – Évaluation déterministe sur le split complet. Robustesse : les deux modèles battent le B&H sur 5/5 sous-fenêtres aléatoires (Single +9,0 % ± 4,9, Multi +8,1 % ± 3,1).

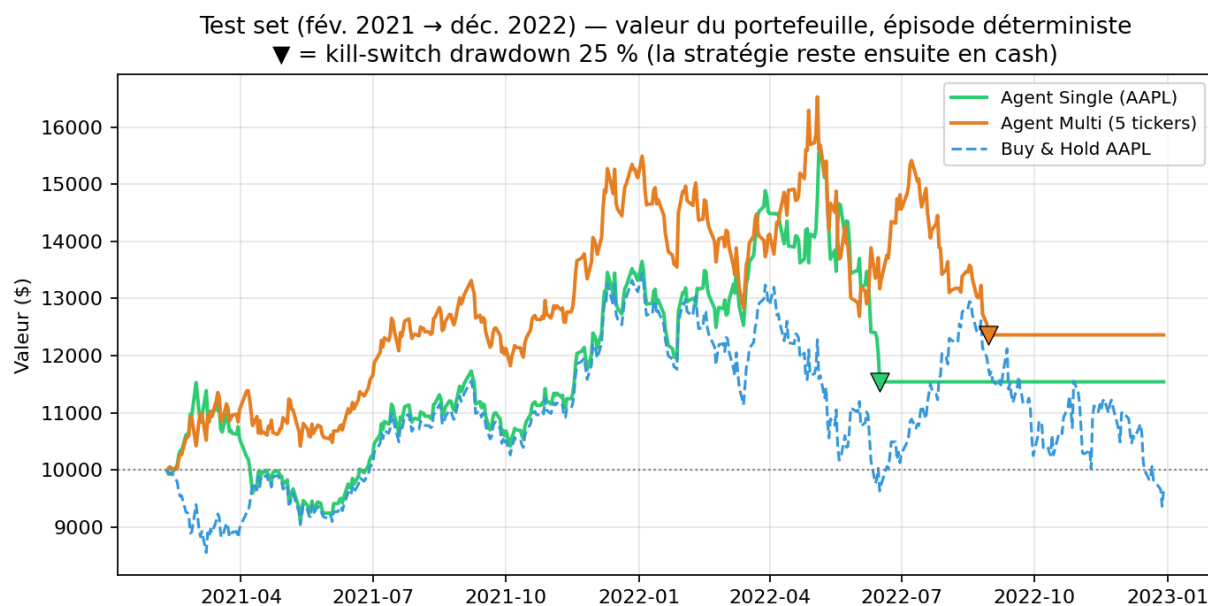


Figure 3 — Valeur du portefeuille sur le test. Lecture : les agents suivent la hausse de 2021 (le Multi la surperforme), encaissent une partie du retournement de 2022, puis le kill-switch les sort du marché (▼) — ils finissent en cash pendant que le B&H replonge. Une part importante de l’alpha vient de cette *non-exposition* finale : vraie décision de gestion du risque, mais il faut le dire — et la section 7 montrera ce que ça implique.

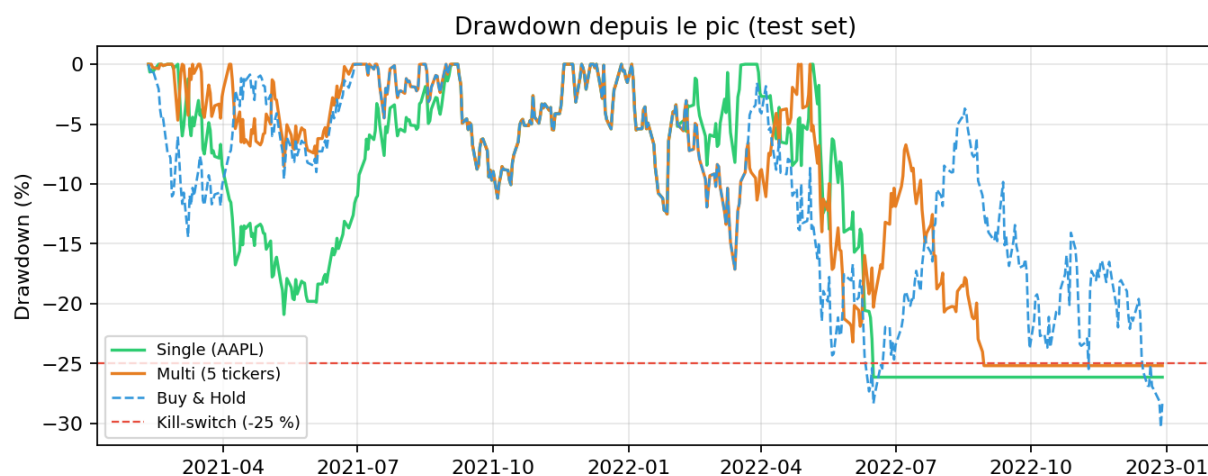


Figure 4 — Drawdown depuis le pic : les agents restent bornés par le kill-switch à -25% quand le B&H dépasse -30% .

6.2 Généralisation cross-actifs

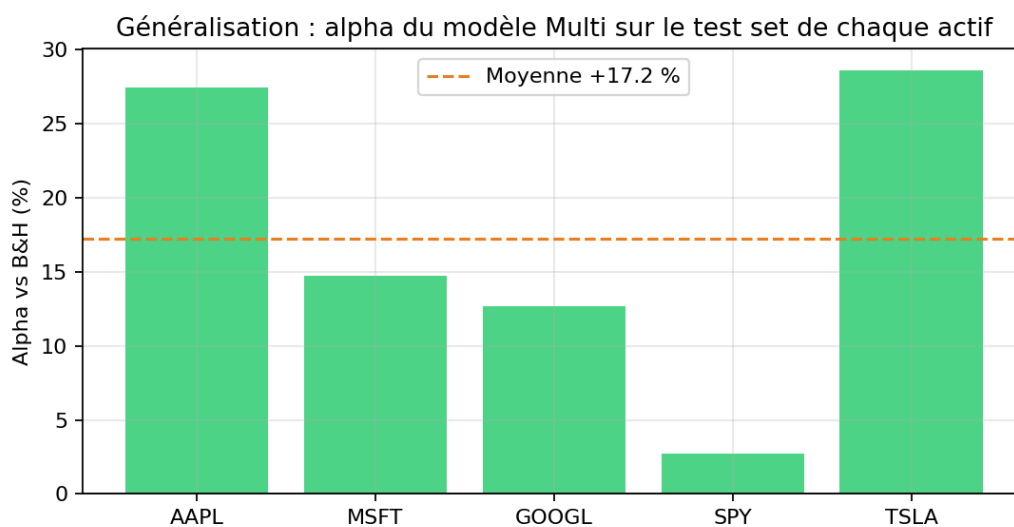


Figure 5 — Alpha du Multi sur le test de chaque actif : positif sur 5/5, moyenne +17,2%. Un modèle qui aurait mémorisé AAPL ne transférerait pas sur TSLA (+28,6%) ni GOOGL (+12,7%).

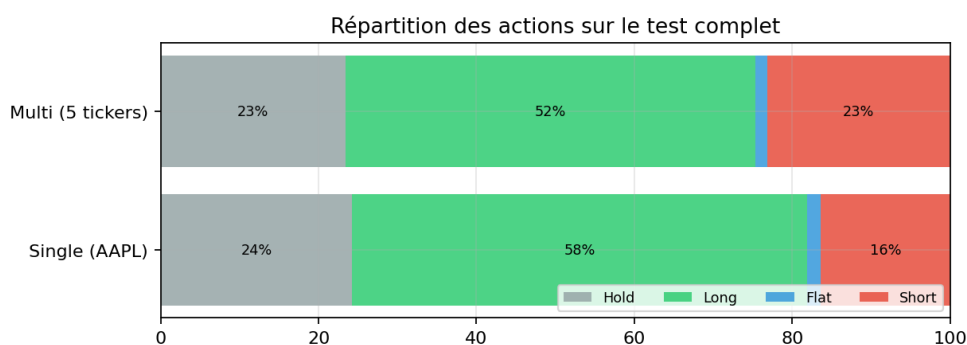


Figure 6 — Répartition des actions sur le test complet. Le Multi est équilibré (short 23% du temps) là où sa version pré-correction était longue en quasi-permanence.

À ce stade, j'avais un résultat solide *sur une période*. Mais je savais — pour l'avoir écrit moi-même dans les limites — qu'un backtest unique est *un tirage*, pas une distribution. La suite s'imposait d'elle-même.

7 Du backtest unique au walk-forward

7.1 Pourquoi c'était la priorité (et pas la diffusion ou le MARL)

Mes lectures me tiraient vers l'avant : politiques par diffusion, génération de scénarios synthétiques, multi-agents. Mais je me suis imposé trois questions avant de choisir :

1. *Ma conclusion actuelle est-elle fiable ?* Non — elle repose sur une seule période de test, choisie par moi.
2. *Qu'est-ce qu'un professionnel me demanderait en premier ?* « Montre-moi la stabilité temporelle » — pas « ajoute un modèle de diffusion ».
3. *Que vaut une sophistication bâtie sur une évaluation fragile ?* Rien : je ne saurais même pas dire si elle améliore quoi que ce soit.

Décision : consolider la fondation d’abord. Le *walk-forward* est le standard du backtesting sérieux : on réentraîne le modèle en ne voyant que le passé, on le teste sur l’année suivante, on avance, on répète. C’est la généralisation dynamique du principe out-of-sample de l’économétrie : non pas *un* échantillon de test, mais une *séquence* d’échantillons de test, chacun prédit par un modèle qui ne connaît que son passé.

7.2 Protocole

Fenêtre *croissante ancrée* (anchored) : pour chaque année de test $Y \in \{2018, \dots, 2022\}$, j’entraîne sur $[2010, Y)$ (dont les derniers 15 % servent de validation pour la sélection du best model), puis j’évalue sur l’année Y — 100 % out-of-sample — pour chacun des cinq actifs. Soit 5 réentraînements complets et $5 \times 5 = 25$ cellules année×actif. Deux limites assumées : 400 000 pas d’entraînement par fold (contre 800 000 pour le modèle de référence, pour un temps de calcul raisonnable), et des folds anciens qui voient moins d’histoire.

7.3 Ce que j’avais prédit — et ce qui s’est passé

Avant de lancer, j’ai noté ma prédiction : *l’alpha moyen serait nettement plus faible que le +27,5 % du split unique — 2022 étant l’année rêvée pour un agent défensif — mais je m’attendais à une médiane positive*. La première moitié s’est vérifiée. La seconde non : médiane $-2,0 \%$, 48 % de cellules positives. Et c’est la moitié fausse qui m’a le plus appris.

Année OOS	Agent (moy.)	B&H (moy.)	Alpha	Régime
2018	+5,2 %	−4,0 %	+9,2 %	année baissière
2019	+21,2 %	+42,2 %	−21,0 %	bull soutenu
2020	−3,0 %	+143,3 %	−146,4 %	krach puis rebond violent
2021	+32,2 %	+43,3 %	−11,1 %	bull
2022	+6,2 %	−31,4 %	+37,6 %	bear
Agrégat : alpha moyen $-26,3 \%$ (médiane $-2,0 \%$, écart-type 106 %), 48 % de cellules > 0 .				

Table 6 — Walk-forward, moyennes par fold sur les 5 actifs. La moyenne est écrasée par 2020 — en particulier la cellule TSLA (B&H +576 %, alpha -514%) : rater UNE année de rebond extrême coûte plus que tout ce que la défense rapporte ailleurs.

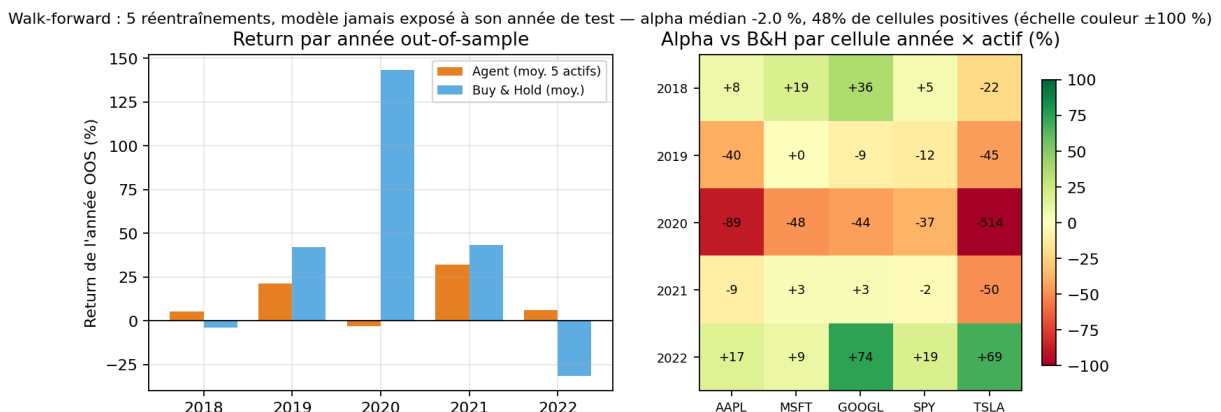


Figure 7 — À gauche : return absolu de l’agent (orange) vs B&H (bleu) par année out-of-sample. À droite : grille des 25 alphas (couleur tronquée à $\pm 100 \%$). Le motif est net : vert sur les lignes 2018 et 2022 (marchés baissiers), rouge sur 2019–2021 (marchés haussiers).

7.4 Interprétation financière : un profil, pas un alpha universel

- **L’agent est un profil défensif régime-dépendant.** Alpha nettement positif les années baissières (2018 : +9 % ; 2022 : +38 %), proche de zéro à négatif en marché haussier, catastrophique — *relativement au B&H* — dans un rebond violent (2020 : stoppé dans le krach, il reste défensif pendant que le marché fait +143 %).
- **En absolu, le tableau est différent** : rendement positif 4 années sur 5, et +5 à +6 % les deux années où le B&H perd. Comme brique *absolute return* défensive dans une allocation, le profil est cohérent ; comme stratégie « qui bat le marché », non.
- **Mon +27,5 % du split unique n’était donc pas faux, mais partiel** : 2021–2022 contient précisément le régime où ce profil excelle. C’est la démonstration empirique, sur mon propre projet, du biais de période — je l’avais énoncé en théorie à l’itération 3, le walk-forward le quantifie.
- **Moyenne vs médiane** : l’écart (−26 % vs −2 %) vient d’une queue gauche épaisse (TSLA 2020). Réflexe d’économetre : sur des distributions asymétriques à outliers, la médiane est la statistique robuste — mais ici l’outlier n’est pas une erreur de mesure, c’est un vrai scénario de marché : le coût d’opportunité des rebonds fait partie du risque de la stratégie.

7.5 Ce que ça ouvre

Le diagnostic « défensif partout, y compris quand il ne faudrait pas » pointe vers une cause précise : l’agent n’observe rien qui lui permette de *distinguer les régimes* — il applique la même prudence dans un krach durable et dans un rebond en V. Trois pistes, par ordre de priorité que je me fixe :

1. **features de régime** (tendance longue, distance au plus-haut d’un an, vitesse de reprise — l’esprit des modèles à changement de régime de type Markov-switching, vus en économétrie) ;
2. **position continue** (action $\in [-1, 1]$) au lieu du tout-ou-rien, pour que la défense soit graduelle et non binaire ;
3. **adversarial training multi-agents** (section 3.3) et génération de scénarios par diffusion — ma roadmap initiale — qui ne prennent leur sens que maintenant : j’ai enfin un protocole capable de mesurer si elles améliorent la robustesse.

8 Stress-tests : les questions qu’un desk poserait

Pour juger mon propre travail, je me suis mis dans la peau d’un trader qui lit ce rapport. Ses deux premières questions sont prévisibles — alors autant y répondre avec des mesures plutôt que des arguments. Aucun réentraînement ici : je stresse les *hypothèses d’exécution et de risque*, à politique fixée (`evaluate.stress_report()`).

8.1 « Ton alpha survit-il à des frais réalistes ? »

Mes modèles sont entraînés avec 0,10 % de frais par trade. J’évalue les mêmes politiques sous une grille de coûts — une façon de proxyser spread, slippage et (partiellement) le coût d’emprunt des shorts que mon simulateur ne modélise pas :

Alpha (test)	0 bp	5 bp	10 bp	20 bp	30 bp
Single (AAPL), 259 trades	+24,9 %	+22,0 %	+19,2 %	+13,8 %	+8,7 %
Multi (5 actifs), 301 trades	+28,7 %	+28,0 %	+27,5 %	+25,7 %	+ 23,7 %

Table 7 – Sensibilité de l’alpha au niveau de frais par trade (test AAPL 2021–2022). 1 bp (point de base) = 0,01 % ; 10 bp = niveau d’entraînement.

Réponse : l'edge du Multi n'est pas un artefact de coûts — il ne perd que 5 points d'alpha entre 0 et 30 bp, soit trois fois le niveau de frais auquel il a été entraîné. Le Single, lui, se dégrade nettement (−16 points sur la grille) : son avantage est plus fragile à l'exécution.

8.2 « Que reste-t-il si j'enlève ton kill-switch ? »

La figure 3 suggérerait qu'une part importante de l'alpha 2022 venait de la sortie du marché sur stop. Je l'ai quantifiée : mêmes politiques, drawdown libre (seul le stop de quasi-ruine à 5 % du capital subsiste).

Sans kill-switch	Return	Alpha	MaxDD	Contribution du stop
Single (AAPL)	−5,4 %	−1,7 %	44,9 %	+21,0 points
Multi (5 actifs)	+1,4 %	+5,1 %	40,4 %	+22,3 points

Table 8 — Ablation du kill-switch sur le test. « Contribution du stop » = alpha avec stop − alpha sans stop (la règle inclut la non-réexposition jusqu'à la fin de la fenêtre).

La réponse est la découverte la plus importante de ce projet après le walk-forward : **environ 21 points d'alpha sur 2022 viennent de la règle de stop, pas du timing quotidien**. Sans elle, le Single ne bat plus le marché du tout (−1,7 %) ; le Multi conserve un alpha de décision modeste mais réel (+5,1 %), en subissant un drawdown de 40 %.

Interprétation en langage de desk : je distingue désormais *l'alpha de signal* (les décisions jour par jour — faible mais positif pour le Multi) de *l'alpha de règle de risque* (couper l'exposition sur drawdown — majoritaire ici). Ce n'est pas disqualifiant : le stop fait partie de la stratégie que j'ai conçue, et c'est lui qui borne le MaxDD à 25 % quand le marché fait −30 %. Mais vendre ce système comme du « market timing par RL » serait faux : c'est *une discipline de risque systématique, aidée par un signal RL modeste*. Cette phrase résume honnêtement le projet.

8.3 Les questions auxquelles je n'ai pas encore de mesure

- *Sensibilité au seed d'entraînement* : tous mes modèles partagent le seed 42 ; il faudrait ré-entraîner sur plusieurs seeds pour donner un intervalle sur l'alpha (coût : quelques heures de calcul).
- *Coût d'emprunt des shorts* : non modélisé explicitement (le stress de frais le capture partiellement) ; sur des titres chers à emprunter, l'alpha short de 2022 serait rogné.
- *Exécution au cours de clôture* : pas d'écart d'exécution intrajournalier ni d'impact de marché.

9 Overfitting, underfitting, régimes : le diagnostic complet

L'overfitting se voit sur le train. Le Single réalise sur son propre train un alpha de l'ordre de +16 500 % : de la **mémorisation** de trajectoire, l'équivalent d'un R^2 in-sample de 0,999. Ce chiffre ne mesure rien d'autre que la capacité du réseau à apprendre par cœur — ce qui compte est out-of-sample.

Mes remèdes. Entraînement multi-actifs (régularisation par les données) ; *early stopping* — on conserve la version du réseau qui maximise la performance de *validation*, pas la dernière : si la fin de l'entraînement sur-apprend, on ne la paie pas (d'où le caractère critique du bug de la section 5.5) ; réseau volontairement petit ; clipping PPO ; bonus d'entropie ; plus d'années de données.

Et l'underfitting ? C'est le défaut symétrique : un modèle trop simple pour capter le signal disponible, reconnaissable à de mauvaises performances *partout, y compris sur le train*. Mon alpha d'entraînement délirant prouve que la capacité ne manque pas. En revanche, le walk-forward révèle un underfitting plus subtil : un underfitting de *représentation*. Mes cinq features ne contiennent tout simplement pas l'information « dans quel régime de marché sommes-nous ? » — aucun réseau, aussi gros soit-il, ne peut apprendre à distinguer un krach durable d'un rebond en V s'il ne voit que dix jours de features court-terme. Le problème n'est pas la capacité du modèle, c'est ce que je lui donne à regarder.

La distinction que le walk-forward m'a forcé à faire. Avant, je risquais de tout appeler « overfitting ». Maintenant je sépare trois choses :

- **mémorisation** (train $\gg$ test) : présente, contenue par les remèdes ci-dessus, et visible ;
- **biais de période** (un test favorable au style du modèle) : c'était mon cas, quantifié par le walk-forward ;
- **dépendance de régime** (une stratégie dont la performance est conditionnelle à l'état du marché) : ce n'est pas un défaut d'apprentissage, c'est une caractéristique du produit — à documenter, couvrir ou conditionner, comme pour n'importe quelle stratégie systématique.

10 Limites honnêtes et travaux futurs

- **Profil régime-dépendant** (section 7) : la limite principale désormais, avec ses trois pistes de réponse (features de régime, position continue, adversarial training).
- **L'alpha est majoritairement porté par la règle de stop** (section 8) : sans kill-switch, il reste +5 % (Multi) à -2 % (Single). L'agent n'a pas appris une gestion du risque continue — le MaxDD $\approx$ 25 % EST le seuil du stop.
- **Sharpe** < 1 : honorable pour du mono-actif avec frais, loin des standards d'un desk — le cadre sans allocation limite structurellement la diversification (Markowitz).
- **Biais de sélection des actifs** : AAPL, MSFT, GOOGL, SPY, TSLA sont des survivants à succès — biais du survivant assumé, à corriger avec un univers plus large et daté.
- **Microstructure simplifiée** : frais constants, pas de slippage ni d'impact, exécution au cours de clôture, emprunt de titres gratuit — la grille de frais de la section 8 borne partiellement ces effets (le Multi tient jusqu'à 30 bp).
- **Walk-forward simplifié** : 400k pas par fold, folds annuels, pas d'agrégation en portefeuille multi-actifs.

À plus long terme, la feuille de route du dépôt ([roadmap.md](#)) prévoit le multi-agents (adversaire de marché, MAPPO — section 3.3) et la génération de scénarios par modèles de diffusion ; mes lectures récentes (survey des modèles de diffusion pour le RL, survey de l'apprentissage de représentations d'état) alimenteront ces phases — avec, cette fois, un protocole d'évaluation digne de mesurer leur apport.

11 Glossaire minute

Chaque concept en trois temps : la définition, le rôle qu'il joue *dans ce projet*, et — quand il y en a un — le piège associé. C'est ma fiche de révision d'avant-entretien.

Alpha Surperformance par rapport à une référence. Formellement, l'alpha de Jensen retranche la part du rendement expliquée par l'exposition au marché : $\alpha_J = R_p - [R_f + \beta(R_m - R_f)]$. Ici je prends $\beta = 1$ et $R_f = 0$, donc $\alpha = R_p - R_{B\&H}$: c'est à la fois ma récompense d'entraînement (par pas) et ma métrique d'évaluation (par période). Piège : un alpha ne

veut rien dire sans préciser la référence ET la fenêtre — tout mon chapitre walk-forward tient dans cette remarque.

Avantage (A_t) Ce qu’une action a rapporté *au-delà* de ce que la fonction de valeur attendait de l’état : $A(s, a) = Q(s, a) - V(s)$. C’est le signal que le policy gradient renforce — pas le rendement brut, l’écart à l’attente. Estimé ici par GAE avec $\lambda = 0,95$ (annexe A.2), l’arbitrage biais/variance appliqué au gradient.

Buy & Hold (B&H) Acheter le premier jour, ne plus rien faire. La référence passive que toute stratégie active doit justifier de battre *après frais* — car le B&H, lui, ne paie qu’un seul aller. Sur mon test 2021–2022 : $-3,7\%$, avec un drawdown de 30% : battre le B&H ne veut pas dire gagner beaucoup, parfois juste perdre moins.

Coûts de friction Tout ce qui sépare un backtest du réel. *Spread* : écart entre prix acheteur et vendeur — on paie la moitié à chaque passage. *Slippage* : écart entre le prix décidé et le prix exécuté. *Impact* : mes propres ordres déplacent le prix (négligeable ici, pas en desk). *Emprunt de titres* : shorter exige d’emprunter l’action, moyennant un loyer qui peut être prohibitif sur les titres demandés. Mon simulateur résume tout en frais proportionnels — la grille de stress (section 8) teste jusqu’à 30 bp.

CVaR₉₅ / VaR₉₅ La VaR₉₅ est le quantile à 5% des rendements journaliers : le seuil de perte franchi en moyenne un jour sur vingt. La CVaR (Expected Shortfall) répond à la question suivante : *ces jours-là, combien perd-on en moyenne ?* Elle regarde *dans* la queue, pas seulement où elle commence, et elle est sous-additive (diversifier ne peut pas l’augmenter) — ce que la VaR ne garantit pas ; d’où son adoption réglementaire (FRTB).

Drawdown Perte courante depuis le dernier pic : $DD_t = (\max_{s \leq t} V_s - V_t) / \max_{s \leq t} V_s$. Le *max drawdown* en est le pire cas sur la période. C’est le risque « vécu » : un investisseur ne ressent pas une variance, il ressent -25% depuis le plus haut. Mon kill-switch est défini dessus, et le Calmar rapporte le rendement à lui.

Early stopping Conserver la version du modèle la meilleure *en validation* plutôt que la dernière. C’est une régularisation : si la fin de l’entraînement sur-apprend le train, on ne la paie pas. Corollaire qui m’a coûté cher : la qualité de cette sélection vaut celle du signal de validation (tableau 3 : $+30$ points d’alpha entre une validation corrompue et une propre, tout le reste identique).

Entropie (bonus d’) $\mathcal{H}[\pi] = -\sum_a \pi(a) \log \pi(a)$, maximale quand la politique hésite uniformément. Un petit bonus ($c_2 = 0,05$) récompense cette hésitation résiduelle → l’agent continue d’explorer au lieu de se figer prématurément sur « toujours Hold » ou « toujours Long ». Trop d’entropie : l’agent ne converge jamais ; pas assez : il converge sur la première stratégie passable.

Épisode Une traversée de la simulation, du point de départ jusqu’à la fin des données (*truncated*) ou au déclenchement d’un stop (*terminated*). À l’entraînement, les départs sont aléatoires pour varier l’expérience ; à l’évaluation de référence, le départ est fixe.

Épisode déterministe (full-split) Départ fixe + politique déterministe (l’action la plus probable, sans tirage) : deux exécutions donnent exactement le même résultat, au centime. C’est mon évaluation de référence — la version « fenêtres aléatoires » ne sert plus qu’à mesurer la robustesse au point d’entrée.

Fonction de valeur $V(s)$ L’estimation, par le critique, de ce qu’un état rapportera en moyenne sous la politique courante. C’est l’« attente » dont l’avantage mesure l’écart, et la baseline qui réduit la variance du gradient sans le biaiser (annexe A.1).

Kill-switch Arrêt de l’épisode si le drawdown dépasse 25% — l’équivalent d’un stop-loss de mandat institutionnel. Mon ablation (section 8) montre qu’il porte ≈ 21 points de mon

alpha 2022 : c'est une décision de *design* de la stratégie, pas un détail d'implémentation, et je dois le dire quand je présente mes chiffres.

Look-ahead bias Utiliser à la date t une information indisponible en t . Les formes sournoises : normaliser avec des statistiques calculées sur tout l'échantillon, sélectionner les actifs *a posteriori* (survivants), choisir ses hyperparamètres en regardant le test. Le péché capital du backtesting — verrouillé ici par des tests unitaires sur le scaler et des splits strictement chronologiques.

MDP Processus de décision markovien : état $\rightarrow$ action $\rightarrow$ récompense $\rightarrow$ état suivant, avec la propriété que l'état résume tout le passé utile. Les marchés violent cette propriété (10 jours de features ne résument pas un régime) — l'hypothèse est une approximation de travail, et sa violation est exactement mon « underfitting de représentation ».

Out-of-sample Évalué sur des données jamais utilisées ni pour entraîner, ni pour choisir le modèle ou ses hyperparamètres. Le seul chiffre qui compte, et la hiérarchie est stricte : train < validation < test < walk-forward (une *séquence* de tests).

Overfitting / underfitting Apprendre par cœur le train (excellent in-sample, mauvais out-of-sample) / ne pas capter le signal disponible (mauvais partout). Mon train à +16 500 % d'alpha crie l'overfitting de mémorisation ; mais mon vrai plafond est un *underfitting de représentation* : aucune capacité de réseau ne compense des features qui ne contiennent pas l'information de régime.

Point de base (bp) 0,01 %. L'unité standard des coûts d'exécution et des écarts de taux : dire « 10 bp par trade » est plus précis et plus professionnel que « 0,1 % ».

Policy gradient / PPO Optimiser directement la politique par montée de gradient sur l'avantage, sans passer par une table de valeurs. PPO y ajoute le clipping du ratio ρ_t dans $[1 - \varepsilon, 1 + \varepsilon]$: aucune mise à jour ne peut trop déplacer la politique, quel que soit le lot d'expérience — la stabilité avant la vitesse, indispensable sur données bruitées (perte complète en annexe A.3).

Seed Graine du générateur pseudo-aléatoire. La fixer rend l'expérience exactement reproductible (comparer deux versions du code à hasard identique) ; la faire *varier* mesure la sensibilité au hasard — ce que je n'ai pas encore fait pour l'entraînement, et que je liste honnêtement dans les questions ouvertes.

Sharpe / Sortino / Calmar Trois façons de rapporter le rendement au risque : à la volatilité totale ($\bar{r}/\hat{\sigma} \times \sqrt{252}$ — les 252 jours de bourse annualisent), à la volatilité *baissière* seule (Sortino ne pénalise pas les bonnes surprises), au max drawdown (Calmar). Un desk regarde les trois : un bon Sharpe avec un Calmar médiocre signale des pertes concentrées.

VecNormalize Standardisation *glissante* des observations pendant l'entraînement (moyenne et variance mises à jour en continu). Ses statistiques font partie du modèle : les recharger à l'évaluation est obligatoire — prédire sur observations brutes fut le bug le plus répété de ce projet (deux scripts touchés).

Walk-forward Réentraîner en ne voyant que le passé, tester sur la période suivante, avancer, répéter. Transforme *un* backtest en *distribution* de résultats out-of-sample — et c'est cette distribution qui a requalifié mon +27,5 % en profil défensif régime-dépendant (médiane -2 %, 48 % de cellules positives).

12 Reproduire les résultats

- Entraînement complet : `python train.py` (~10–15 min CPU) ;
- Rapport d'évaluation : `python evaluate.py` ;

- Walk-forward : `python walk_forward.py` (~25 min, 5 folds);
- Suite de tests : `python -m pytest` (67 tests; marqueurs `network` et `slow` pour l'intégration);
- Figures de ce rapport + dashboard statique : `python reports/build_assets.py`;
- Dashboard interactif : `streamlit run dashboard.py`.

Avertissement : projet strictement éducatif. Rien ici ne constitue un conseil d'investissement; les performances simulées ne préjugent de rien.

A Annexe mathématique

Cette annexe déroule les mathématiques que le corps du rapport résume. Rien ici n'est nécessaire pour utiliser le projet; tout est utile pour le *défendre*.

A.1 Du policy gradient à l'avantage

On cherche les paramètres θ du réseau qui maximisent le rendement espéré $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [\sum_t \gamma^t r_t]$, où τ est une trajectoire (un épisode). Le *théorème du policy gradient* donne :

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) Q^\pi(s_t, a_t) \right],$$

où $Q^\pi(s, a)$ est la valeur d'une action dans un état. L'astuce de dérivation — dite du *ratio de vraisemblance* — parlera à un économètre : c'est la même identité $\nabla p = p \nabla \log p$ qui fait apparaître le score dans le maximum de vraisemblance. Intuition : on augmente la log-probabilité des actions proportionnellement à ce qu'elles ont rapporté.

Problème : Q est estimé par des rendements réalisés, extrêmement bruités en finance $\Rightarrow$ variance énorme du gradient. Remède : soustraire une *baseline* $b(s_t)$ qui ne dépend pas de l'action. Ce retranchement ne biaise pas le gradient car

$$\mathbb{E}_{a \sim \pi} [\nabla_\theta \log \pi_\theta(a | s) b(s)] = b(s) \nabla_\theta \underbrace{\sum_a \pi_\theta(a | s)}_{=1} = 0.$$

Le choix optimal en pratique est $b(s) = V^\pi(s)$ (la fonction de valeur), ce qui fait apparaître l'**avantage** :

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \quad : \text{ce que l'action a rapporté } \textit{au-delà} \text{ de l'attente de l'état.}$$

A.2 Estimation de l'avantage : GAE

Comment estimer A_t ? Deux extrêmes : le résidu de différence temporelle $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ (peu de variance, mais biaisé si V est mal estimée) ou le rendement Monte-Carlo complet (sans biais, variance maximale). GAE (*Generalized Advantage Estimation*) interpole entre les deux par une somme géométrique de résidus :

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}, \quad \lambda \in [0, 1].$$

$\lambda = 0$ redonne δ_t (tout biais), $\lambda = 1$ le Monte-Carlo (toute variance). J'utilise la valeur standard $\lambda = 0,95$: sur des rendements financiers, accepter un peu de biais pour beaucoup moins de variance est le bon échange — exactement l'arbitrage biais/variance du cours d'économétrie, appliqué à l'estimation d'un gradient.

A.3 La perte PPO complète

Ce que Stable-Baselines3 minimise réellement est la somme de trois termes :

$$\mathcal{L}(\theta) = \underbrace{-\mathbb{E}_t \left[\min(\rho_t A_t, \text{clip}(\rho_t, 1 - \varepsilon, 1 + \varepsilon) A_t) \right]}_{\text{politique (clippée)}} + c_1 \underbrace{\mathbb{E}_t [(V_\theta(s_t) - V_t^{\text{cible}})^2]}_{\text{valeur}} - c_2 \underbrace{\mathbb{E}_t [\mathcal{H}[\pi_\theta(\cdot | s_t)]]}_{\text{entropie}},$$

avec ici $\varepsilon = 0,15$, $c_1 = 0,5$, $c_2 = 0,05$ et $\rho_t = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$.

Trois lectures :

- le **min** rend l'objectif *pessimiste* : si le ratio ρ_t sort de $[1 - \varepsilon, 1 + \varepsilon]$ dans le sens qui augmenterait l'objectif, le gradient est coupé — on ne « surexploite » jamais un lot d'expérience ;
- le terme de **valeur** entraîne le critique (V_t^{cible} est le rendement observé bootstrappé) ; sans bon critique, pas de bon avantage ;
- l'**entropie** $\mathcal{H}[\pi] = -\sum_a \pi(a) \log \pi(a)$ est maximale pour une politique uniforme : la pénaliser négativement ($-c_2 \mathcal{H}$ dans la perte) revient à *récompenser* l'indécision résiduelle, donc l'exploration.

A.4 Le processus d'Ornstein-Uhlenbeck en détail

L'équation $dX_t = \kappa(\mu - X_t) dt + \sigma dW_t$ se résout explicitement (par la méthode du facteur intégrant $e^{\kappa t}$, comme une EDO linéaire) :

$$X_t = \mu + (X_0 - \mu) e^{-\kappa t} + \sigma \int_0^t e^{-\kappa(t-s)} dW_s.$$

Lectures immédiates :

- **Espérance** : $\mathbb{E}[X_t] = \mu + (X_0 - \mu) e^{-\kappa t}$ — l'écart initial fond exponentiellement ; la *demi-vie* $t_{1/2} = \ln 2 / \kappa$ est le temps pour en effacer la moitié.
- **Variance stationnaire** : $\text{Var}[X_\infty] = \sigma^2 / (2\kappa)$ — le processus fluctue d'autant plus large que le rappel est faible.
- **Discrétisation exacte** au pas Δ :

$$X_{t+\Delta} = \mu(1 - \varphi) + \varphi X_t + \varepsilon_t, \quad \varphi = e^{-\kappa \Delta}, \quad \varepsilon_t \sim \mathcal{N}\left(0, \frac{\sigma^2}{2\kappa}(1 - \varphi^2)\right).$$

C'est un **AR(1)** : le pont exact entre calcul stochastique et économétrie. Estimation : une MCO de $X_{t+\Delta}$ sur X_t donne $\hat{\varphi}$, d'où $\hat{\kappa} = -\ln \hat{\varphi} / \Delta$ et la demi-vie. (Tester $\varphi = 1$ contre $\varphi < 1$, c'est exactement le test de racine unitaire de Dickey-Fuller : un OU est l'alternative stationnaire.)

Usage type en trading : modéliser le *spread* de deux actifs cointégrés, estimer κ et $t_{1/2}$, n'entrer que si la demi-vie est courte devant l'horizon de détention et les frais — le cœur du projet stat-arb décrit dans PROJETS_FUTURS.md.